



CYNA

Gestion des incidents et des bugs

Projet fil rouge — Plateforme Cyna

**Adrien CACHOUX · Romain CANER ·
Ethan MIRBEAU**

Version 1.0 — Bloc 3

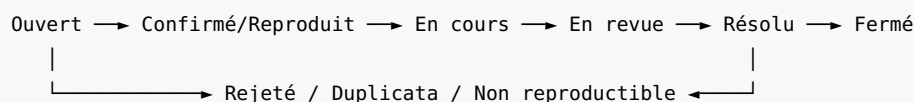
Solutions de cybersécurité SaaS — SOC · EDR · XDR

Ce document décrit le **suivi des bugs**, la **classification des incidents** et le **traitement des erreurs récurrentes**. Il répond à l'axe « Gestion des incidents & bugs » du Bloc 3.

1. Outil de suivi (bug tracking)

Le suivi des anomalies s'appuie sur **GitHub Issues** du dépôt du projet, couplé aux pull requests (chaque correctif référence l'issue qu'il ferme via `Fixes #<n>`). Ce choix garantit la traçabilité bout-en-bout : bug signalé → ticket → branche `fix/*` → revue → CI verte → fusion → issue fermée.

1.1 Cycle de vie d'un ticket



1.2 Modèle de ticket de bug

```
**Description** : (comportement observé)
**Étapes de reproduction** : 1. ... 2. ... 3. ...
**Résultat attendu** :
**Résultat obtenu** :
**Environnement** : local / production · navigateur / version
**Sévérité** : bloquant / majeur / mineur / cosmétique
**Captures / logs** :
```

2. Classification par sévérité

Sévérité	Définition	Délai de prise en charge cible
S1 — Bloquant	Service indisponible, perte de données, faille de sécurité	Immédiat (hotfix)
S2 — Majeur	Fonction clé KO sans contournement (paiement, connexion)	< 24 h
S3 — Mineur	Fonction dégradée avec contournement	Prochain sprint
S4 — Cosmétique	Défaut visuel, texte, sans impact fonctionnel	Backlog

Les libellés GitHub correspondants : `bug` , `severity:S1 ... S4` , `security` , `regression` , `duplicate` , `wontfix` .

3. Traitement des erreurs récurrentes

3.1 Détection

Les erreurs récurrentes sont identifiées à partir :

- des **logs applicatifs** (occurrences répétées d'une même exception — cf. `SUPERVISION_LOGS.md`) ;
- des **erreurs 5xx** relevées dans les logs du reverse proxy ;
- des **retours du formulaire de contact** (back-office).

3.2 Procédure

1. **Regrouper** les occurrences d'une même erreur (signature = message + emplacement).
2. **Prioriser** selon la fréquence × sévérité.
3. **Corriger la cause racine** (pas seulement le symptôme) et **ajouter un test de non-régression** (PHPUnit/JUnit) qui échouait avant le correctif.
4. **Vérifier en CI** puis déployer (cf. `PIPELINE_CICD.md`).
5. **Documenter** dans le registre d'incidents (§4).

Principe : **un bug corrigé = un test ajouté**. La suite de tests ne fait que croître, ce qui prévient les régressions.

4. Registre d'incidents (production)

Journal des incidents significatifs survenus en production (à compléter au fil de l'exploitation) :

#	Date	Sévérité	Description	Cause racine	Résolution	Test ajouté
—	—	—	—	—	—	—

Pour chaque incident S1/S2, rédiger un court **post-mortem** (sans recherche de responsable) : chronologie, impact, cause racine, actions correctives et préventives.

5. Erreurs applicatives — comportement par défaut

- En **production** (`APP_ENV=production`), les exceptions inattendues sont **journalisées** (`error_log`) et l'utilisateur voit une **page 500 générique** (aucune fuite de détail technique).
- En **local**, la trace complète est affichée pour faciliter le débogage.
- Les erreurs métier attendues (`HttpException` : 403, 404...) rendent une page d'erreur dédiée.

Référence code : `Cyna/src/Core/Kernel.php` (conversion des exceptions en réponses HTTP).

6. Démonstration en soutenance (axe incidents & bugs)

1. Montrer le tableau **GitHub Issues** : tickets ouverts/fermés, libellés de sévérité, lien issue ↔ PR ↔ commit de correctif.
2. Prendre un bug résolu : montrer le **test de non-régression** ajouté.
3. Provoquer une erreur et montrer la **page 500 propre** + la **ligne de log** correspondante.

Références

- Supervision & logs
- Pipeline CI/CD
- Cahier de recettes
- DAT — Dossier d'Architecture Technique